

# Assignment 1: Text Classification

CSC5035Z Natural Language Processing 2026

Due: 5 May at 10pm. Total: 50 marks

**Lecturer:** Dr. Jan Buys ([jan.buys@uct.ac.za](mailto:jan.buys@uct.ac.za))

**TA:** Nomonde Khalo ([KHLNOM004@myuct.ac.za](mailto:KHLNOM004@myuct.ac.za))

The assignment should be done by groups of 2 students. Only one student per group should make the submission on Amathuba.

## 1. Overview

In this assignment you will build, analyse, and compare classical text classification pipelines for intent detection in African languages, using the INJONGO dataset (Yu et al., ACL 2025). You will implement a Naive Bayes classifier from scratch, use an existing Logistic Regression implementation, and design your own feature engineering pipeline — including a hand-implemented BPE tokenizer. The emphasis is on rigorous experimentation and careful analysis of results.

A core learning objective is to engage directly with the data, the models, and the results. Accordingly, you are required to implement key components yourself (Naive Bayes, BPE tokenizer), to run systematic ablation experiments, and to explain your findings in your own words with reference to specific examples from the data. Marks are primarily awarded not for the absolute values of the reported results directly, but rather for evidence of engagement with the experimental process.

## 2. Dataset

INJONGO is a multilingual intent detection and slot-filling benchmark for 16 African languages, created by native speakers across five domains: banking, travel, home, kitchen and dining, and utility. We will use the dataset for intent detection (only) as a multiclass classification problem. The dataset contains 3,200 labelled utterances per language across 40 intent classes, split into training (2,240), development (320), and test (640) sets. The dataset is available at:

<https://github.com/masakhane-io/masakhane-nlu/tree/main/InjongoIntent>

The 16 African languages available are listed below with their ISO language codes. English (eng) is also present but is excluded from this assignment.

| Code | Language | Code | Language |
|------|----------|------|----------|
| amh  | Amharic  | lin  | Lingala  |
| ewe  | Ewe      | lug  | Luganda  |

|     |             |     |          |
|-----|-------------|-----|----------|
| hau | Hausa       | orm | Oromo    |
| ibo | Igbo        | sna | Shona    |
| kin | Kinyarwanda | sot | Sesotho  |
| swa | Swahili     | twi | Twi      |
| wol | Wolof       | xho | isiXhosa |
| yor | Yoruba      | zul | Zulu     |

More information about the dataset is available in the paper: Yu, H., Adelani, D. I., et al. (2025). INJONGO: A Multicultural Intent Detection and Slot-filling Dataset for 16 African Languages. In Proceedings of ACL 2025. <https://aclanthology.org/2025.acl-long.464/>

**Language selection.** Choose any two African languages from the table above. Your choice is entirely free, but you should be able to say something meaningful about the two languages you pick — for example, why they differ typologically, or what you expected to find before running experiments. You must use both languages throughout all parts of the assignment.

**Note on the data.** All experiments must use the provided train/dev/test splits without modification. The development set is for hyperparameter tuning and model selection. The test set is evaluated once only, for your final reported numbers. Do not tune on the test set.

### 3. Assignment Structure

The assignment has five parts. Parts 1–4 build on each other; Part 5 is the written analysis.

#### Part 1: Baselines (5 marks)

Implement two simple baseline classifiers using logistic regression, using an implementation from an existing library (e.g. scikit-learn). You must implement the feature extraction yourself.

**1a. Word unigram baseline.** Tokenize each utterance by splitting on whitespace (lowercased, excluding punctuation). Represent each utterance as a bag-of-words count vector. Train a Logistic Regression classifier on these features.

**1b. Character n-gram baseline.** Represent each utterance as a bag of character n-grams (you choose  $n$ , or a range of  $n$ ; this is a hyperparameter to tune). A character n-gram is  $n$  consecutive characters in the input. Train a Logistic Regression classifier on these features. Character n-grams are a natural fit for morphologically rich languages — reflect on this in your analysis.

For both baselines: Tune the Logistic Regression regularisation strength  $C$  on the development set and report the value selected. Report dev and test accuracy for each. Use the same hyperparameter tuning protocol you will apply throughout the assignment (see Section 4).

## Part 2: BPE Tokenizer (10 marks)

Implement a Byte-Pair Encoding (BPE) tokenizer from scratch, without using any existing BPE library (e.g. tokenizers, sentencepiece, tiktoken). You may use standard Python data structures (Counter, defaultdict, etc.).

Your implementation must include:

- Training: learn merge operations from a corpus by iteratively merging the most frequent adjacent symbol pair.
- Encoding: apply the learned merges (in order) to tokenize new text.
- An end-of-word marker (e.g. `_`) so that word boundaries are represented.
- A configurable vocabulary size (number of merges  $k$ ).

Train your BPE tokenizer on the training set of each language separately. The number of merges  $k$  is a hyperparameter to tune.

**Deliverable.** Submit your BPE implementation as a standalone Python module (`bpe.py`) with clear train and encode functions. It must be possible to import and run your tokenizer independently of the classifier code.

**Tokenizer analysis.** For each language, report example tokenizations of 5 utterances from the training set at  $k = 100, 300$ , and  $500$  merges. Discuss what the differences reveal about the morphology of each language.

## Part 3: Naive Bayes Classifier (10 marks)

Implement a multinomial Naive Bayes classifier from scratch. You may not use scikit-learn or any other library's Naive Bayes implementation. You may use numpy for array operations.

Your implementation must include:

- Estimation of log prior probabilities from training label frequencies.
- Estimation of token log-likelihoods per class using token counts.
- Laplace (add- $\alpha$ ) smoothing with configurable  $\alpha$ .
- Prediction by summing log priors and log likelihoods (to avoid underflow).

Train your Naive Bayes classifier using BPE features (from Part 2). Treat the smoothing parameter  $\alpha$  and the number of BPE merges  $k$  as hyperparameters to tune jointly on the development set.

## Part 4: Feature Engineering and Logistic Regression (10 marks)

Using your BPE tokenizer (Part 2) as a foundation, design a feature engineering pipeline and train a Logistic Regression classifier (using scikit-learn or equivalent). The goal is to improve over the word and character baselines through deliberate feature design.

**Required features.** You must include at minimum:

- BPE token counts (your implementation from Part 2).
- At least two additional feature types of your own design.

*Suggested directions (choose based on your analysis, do not use all):*

- TF-IDF weighting of BPE tokens.
- BPE token bigrams.
- Character n-gram features combined with BPE features.
- Word-length or utterance-length features.
- Prefix or suffix features based on observed morphological patterns.

For each feature type you add, run an ablation: train a model with and without that feature type, report the dev accuracy difference, and justify whether it is worth including. Features should be motivated by observations about the data or the language, not added arbitrarily.

## Part 5: Analysis and Report (15 points)

Write a report (maximum 4 pages including tables and figures) covering the results of the experiments in parts 1—4 as well as the analyses below. Your analysis should be grounded in your specific experimental results — general statements about NLP or machine learning that are not connected to your data will not receive credit.

**Language background (brief).** Introduce the two languages you chose. What do you know about their morphological typology, script, and any features relevant to NLP? What did you expect to find before running experiments?

**Tokenizer analysis.** Discuss what your BPE tokenizer learned for each language. What do the most frequent merges reveal? How does the segmentation change with  $k$ ? Does the tokenizer behave differently for the two languages, and why?

**Model comparison.** Summarise your results in a single table covering all models and both languages. Discuss the pattern: which model performs best, does this vary by language, and what explains the differences? Compare NB and LR specifically: where does each model succeed or fail, and what properties of each classifier explain this?

**Hyperparameter analysis.** For at least one hyperparameter (e.g.  $\alpha$  for NB,  $C$  for LR, or  $k$  for BPE), plot dev accuracy as a function of the hyperparameter value for both languages on the same plot. Explain the shape of the curve: what does a very small or very large value imply for the model?

**Error analysis.** Identify the three intent pairs most frequently confused by your best model on each language. For each confused pair: (i) quote two or three misclassified examples from the test set; (ii) inspect the token distributions your model learned for those intents; (iii) explain why the model is confused. Identify at least one pattern that differs between the two languages.

**Feature engineering.** Which additional features helped and which did not? What does this tell you about what information is useful for intent detection in your chosen languages? If you had more time, what would you try next and why?

## 4. Experimental Protocol

All students must follow this protocol to ensure results are comparable and reproducible.

**Hyperparameter tuning.** All hyperparameter selection must be done on the development set. The test set is evaluated once, after all choices are finalised.

**Suggested ranges for hyperparameter tuning:**

- BPE merges  $k$ : at least 5 values spanning a wide range (e.g. 100, 200, 300, 500, 750).
- NB smoothing  $\alpha$ : at least 6 values (e.g. 0.001, 0.01, 0.1, 0.5, 1.0, 5.0).
- LR regularisation  $C$ : at least 6 values (e.g. 0.01, 0.1, 0.5, 1.0, 5.0, 10.0).
- Character  $n$ -gram range: try at least three settings (e.g. unigrams, (2,3), (2,4), (3,5)).

**Reporting.** For every model variant, report dev accuracy alongside the test accuracy. Never select hyperparameters based on test performance.

**Reproducibility.** Set random seeds where applicable. Your code must produce the same results when re-run. Include a requirements.txt or equivalent.

**Language independence.** Your BPE tokenizer and Naive Bayes classifier must be trained independently for each language — no cross-lingual sharing of parameters or vocabulary.

## 5. Submission

Submit a single zip archive on Amathuba containing:

1. bpe.py — standalone BPE tokenizer module.
2. nb.py — standalone Naive Bayes classifier module.
3. main.py (or a Jupyter notebook) — training, evaluation, and experiment code.
4. report.pdf — your written analysis (Part 5) with the results table.
5. requirements.txt — Python dependencies.

**Code quality.** Code should be readable and documented at the level of a professional project — not over-commented, but with clear function names and a docstring where the interface is non-obvious.

**Collaboration and AI policy.** Discussion of concepts with peers is encouraged; sharing code or analysis is not. You may use AI assistants (e.g. GitHub Copilot, ChatGPT) for syntax help and debugging, but the experimental design, hyperparameter tuning, and written analysis must be your own work. If you use an AI tool, include a brief note (one paragraph) at the end of your report describing how and for what. Submissions where the analysis does not match

the experimental results, or where the reported numbers cannot be reproduced from the submitted code, will be investigated for academic integrity. An implementation that is clearly copied from an external source without attribution will receive zero for that component.